suggest explicitly applying the action filter to the action method (instead of globally, or to a controller) to remind you about that implicit coupling.

With `RecipeService` and `id` in place, it's a case of checking whether the identifier corresponds to an existing `Recipe` entity and if not, setting `context.Result` to `NotFoundResult`. This short-circuits the pipeline and bypasses the action method altogether.

NOTE Remember that you can have multiple action filters running in a single stage. Short-circuiting the pipeline by setting `context.Result` prevents later filters in the stage from running and bypasses the action method execution.

Before we move on, it's worth mentioning a special case for action filters. The `ControllerBase` base class implements `IActionFilter` and `IAsyncActionFilter` itself. If you find yourself creating an action filter for a single controller and want to apply it to every action in that controller, you can override the appropriate methods on your controller instead, as in the following listing.

Listing 22.7 Overriding action filter methods directly on `ControllerBase`

```
public class HomeController : ControllerBase      ❶
{
    public override void OnActionExecuting(       ❷
        ActionExecutingContext context)           ❷
    { }                                           ❷
    public override void OnActionExecuted(        ❸
        ActionExecutedContext context)            ❸
    { }                                           ❸
}
```

❶ Derives from the ControllerBase class